



MIMESYS: Generating Realistic Executable Testing Environments from Resource Usage Traces

OSDI' 26

Donghyun Kim Zichao Hu Joydeep Biswas Aditya Akella Daehyeok Kim

The University of Texas at Austin



TEXAS

The University of Texas at Austin



冯敏远

2026/07/06



- 作者介绍
- 研究背景与动机
- 系统设计
- 实验评估
- 总结与讨论



- 作者介绍
- 研究背景与动机
- 系统设计
- 实验评估
- 总结与讨论

作者简介

Donghyun Kim (金东炫)

<https://donghyun-kim.com/assets/pdf/cv.pdf>



院校：（2020年）浦项科技大学—（2022）首尔国立大学—（2023）德克萨斯大学奥斯汀分校

研究方向：

- machine learning for systems
- synthetic data generation for systems

主要研究：

- [IPDPS' 26] Reforge: Low-Latency Distributed GNN Serving with Selective Embedding Recomputation
- [EMNLP' 25] Large Language Models as Realistic Microservice Trace Generators

导师：

1. AdityaAkella



- Learning Directed Operating System (LDOS)
- the InfraAI Center

<https://www.cs.utexas.edu/~akella/>

2. DaehyeokKim



- abstractions and runtime systems for computing infrastructures across edge and cloud datacenters.

<https://daehyeok.kim/>



- 作者介绍
- **研究背景与动机**
- 系统设计
- 实验评估
- 总结与讨论

研究背景与动机



- 应用在真实生产环境中不是独占运行的，而是和其他 workload 共享机器资源。

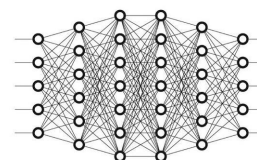
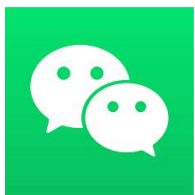
研究背景与动机

- 应用在真实生产环境中不是独占运行的，而是和其他 workload 共享机器资源。



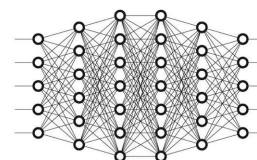
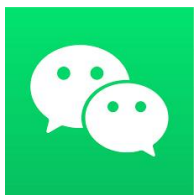
研究背景与动机

- 应用在真实生产环境中不是独占运行的，而是和其他 workload 共享机器资源。



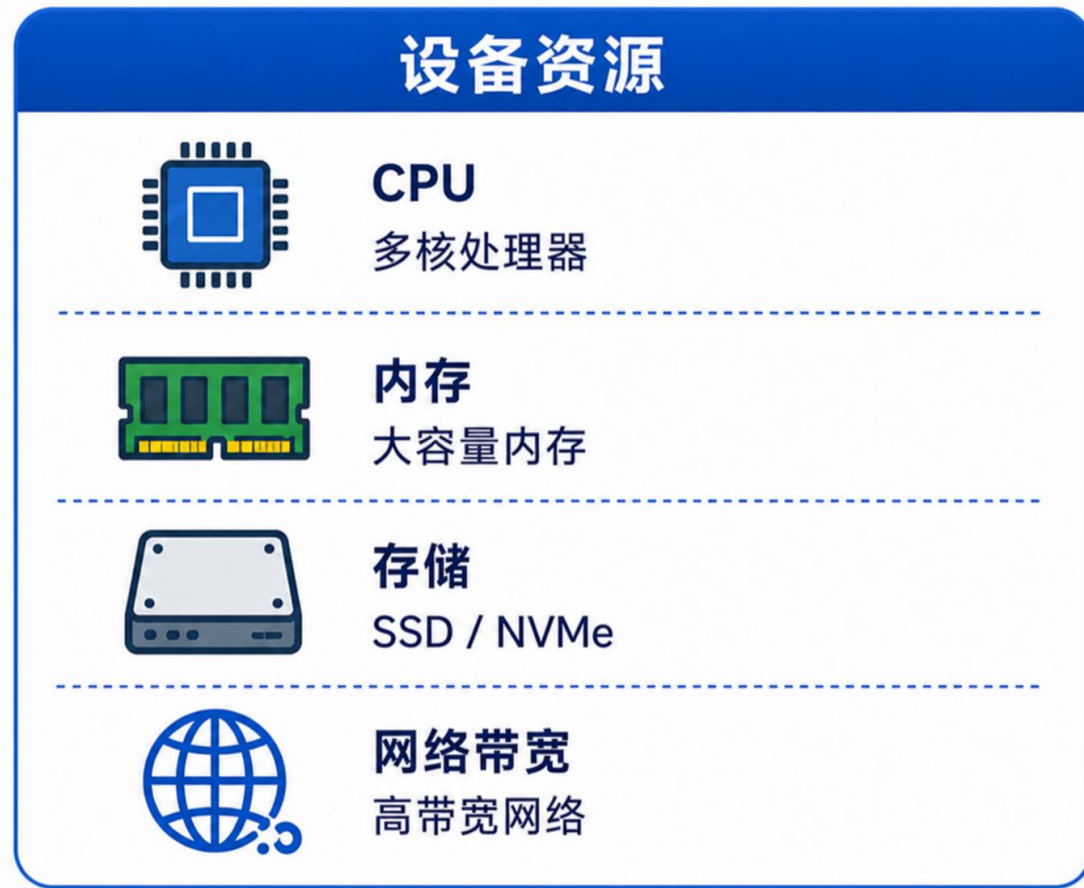
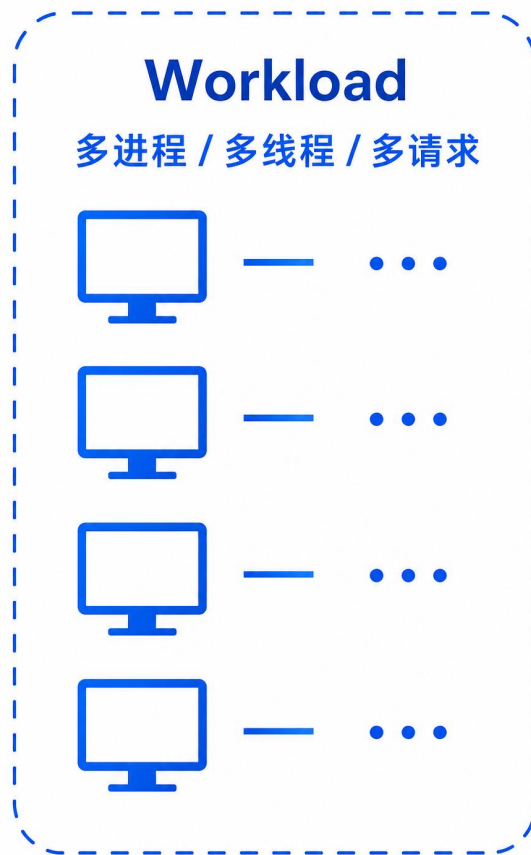
研究背景与动机

- 应用在真实生产环境中不是独占运行的，而是和其他 workload 共享机器资源。

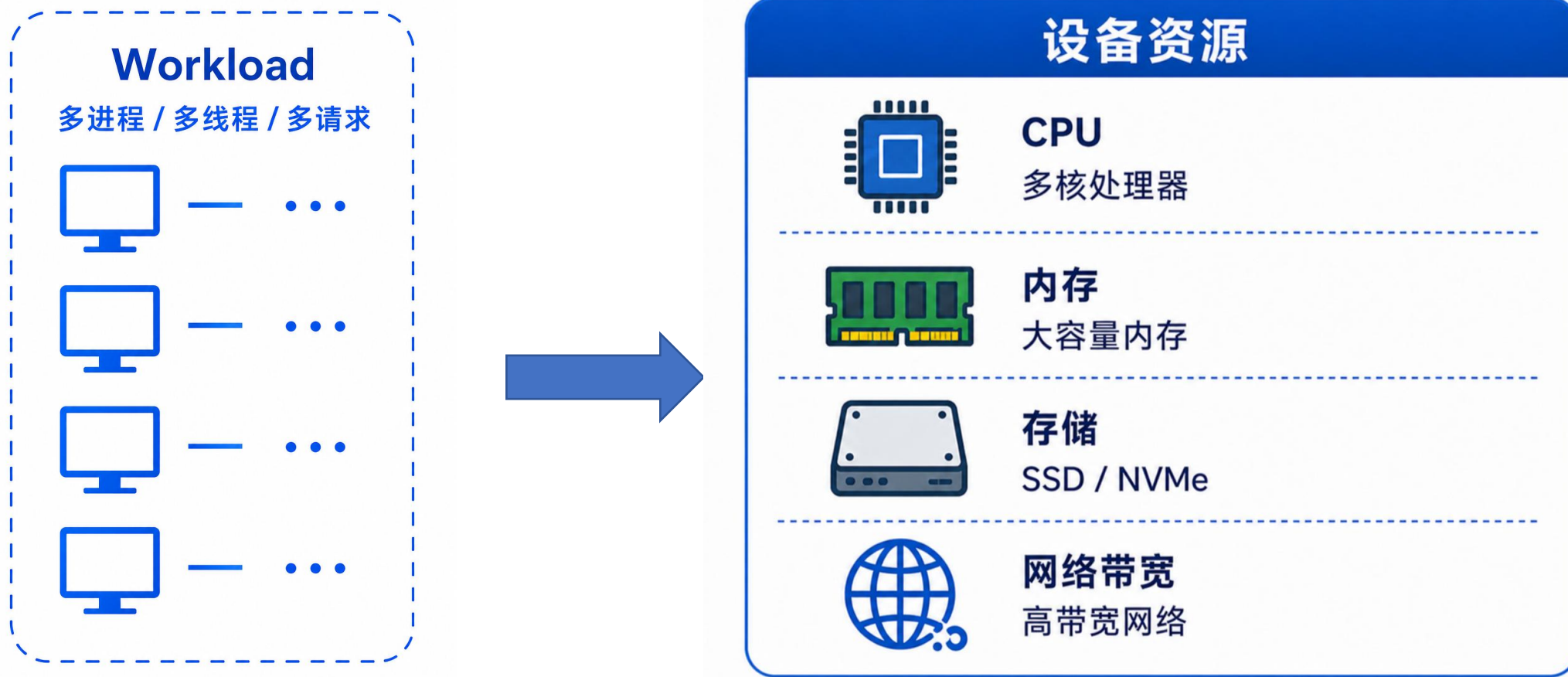


洞见：真实环境里会有资源竞争，一个应用在单独运行时表现很好，不代表它部署到真实云环境后也表现稳定。

研究背景与动机

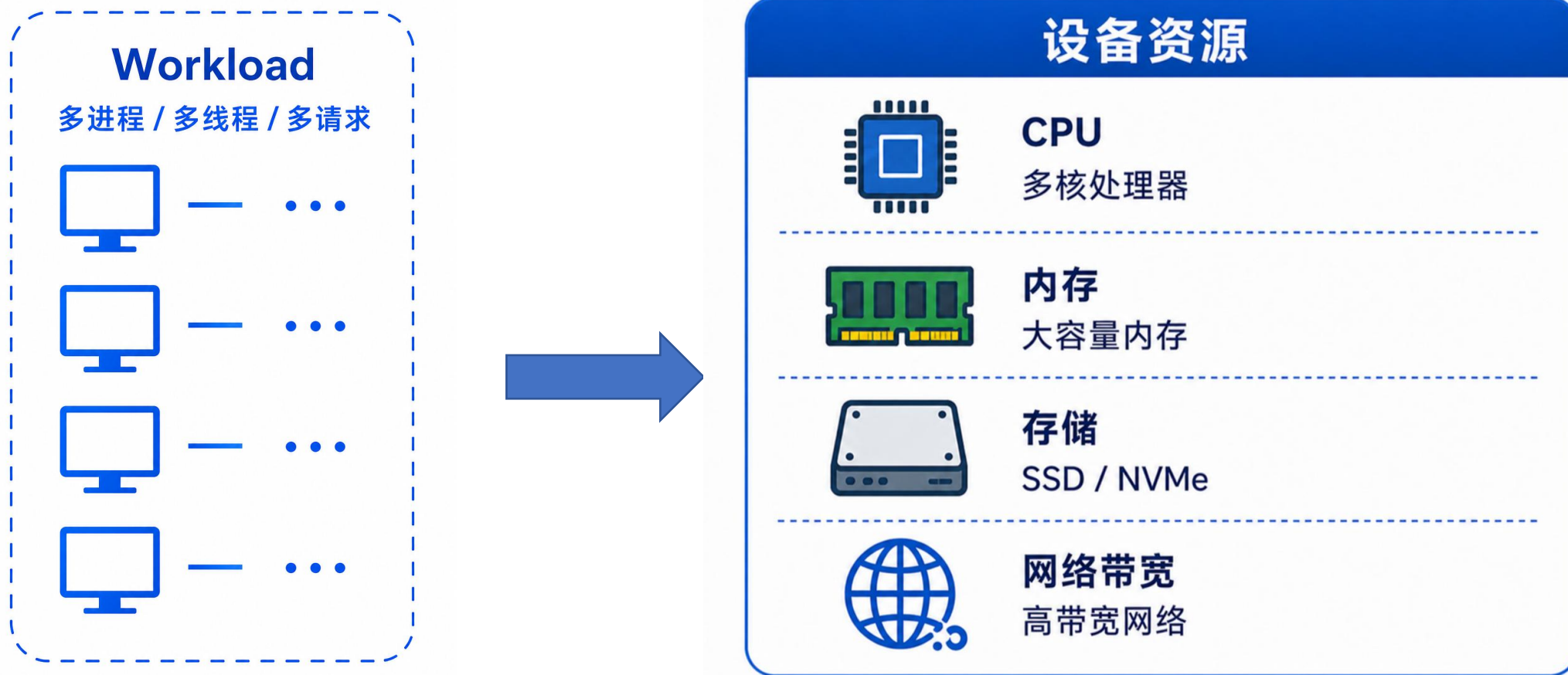


研究背景与动机



- 研究者需要真实的工作负载来评估系统设计、体系结构和优化方法；
- 开发者也需要在接近生产环境的条件下测试软件；
- 集群运维者则需要多样化的 workload 来评估硬件配置和容量规划。

研究背景与动机



- 研究者需要真实的工作负载来评估系统设计、体系结构和优化方法;
- 开发者也需要在接近生产环境的条件下测试软件;
- 集群运维者则需要多样化的 workload 来评估硬件配置和容量规划。

——→ 真实生产环境重要

研究背景与动机



很难拿到真实生产环境中的干扰负载

研究背景与动机

很难拿到真实生产环境中的干扰负载

1. 最直接的做法是直接使用生产环境中的真实 workload，但这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。

研究背景与动机

很难拿到真实生产环境中的干扰负载

1. 最直接的做法是直接使用生产环境中的真实 workload，但这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。
2. 即使开发者能够访问生产环境进行测试，也会遇到两个问题。

研究背景与动机

很难拿到真实生产环境中的干扰负载

1. 最直接的做法是直接使用生产环境中的真实 workload，但这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。

2. 即使开发者能够访问生产环境进行测试，也会遇到两个问题。

第一，**成本高、难以扩展**。开发者需要把应用部署到接近生产环境的基础设施中，才能观察真实资源竞争带来的影响。



研究背景与动机

很难拿到真实生产环境中的干扰负载

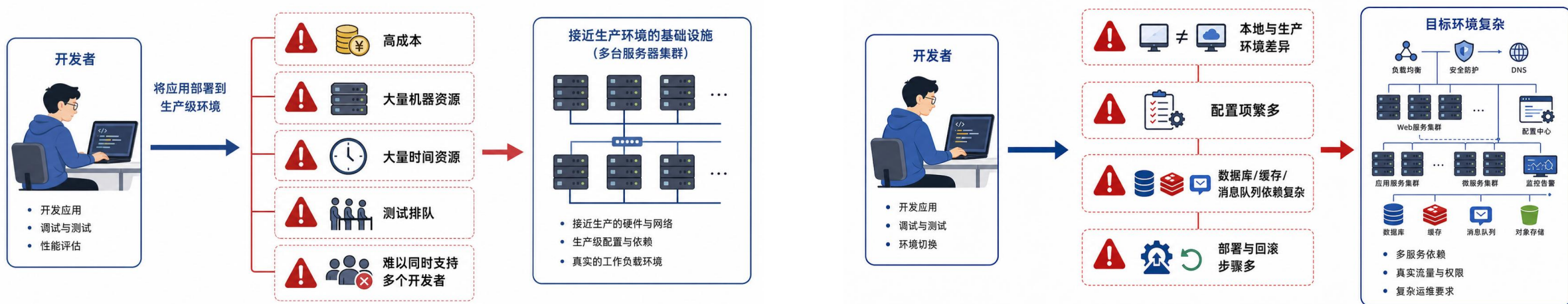
1. 最直接的做法是直接使用生产环境中的真实 workload，但这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。
2. 即使开发者能够访问生产环境进行测试，也会遇到两个问题。
第一，成本高、难以扩展。开发者需要把应用部署到接近生产环境的基础设施中，才能观察真实资源竞争带来的影响。
第二，**生产环境本身是不断变化的**。随着 workload 的变化，资源竞争模式也会变化，测试条件难以复现。



研究背景与动机

很难拿到真实生产环境中的干扰负载

1. 最直接的做法是直接使用生产环境中的真实 workload，这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。
2. 即使开发者能够访问生产环境进行测试，也会遇到两个问题。
第一，**成本高、难以扩展**。开发者需要把应用部署到接近生产环境的基础设施中，才能观察真实资源竞争带来的影响。
第二，**生产环境本身是不断变化的**。随着 workload 的变化，资源竞争模式也会变化，测试条件难以复现。



研究背景与动机

很难拿到真实生产环境中的干扰负载

1. 最直接的做法是直接使用生产环境中的真实 workload，但这在实际中很难做到。真实租户 workload 通常涉及隐私、商业机密和专有业务逻辑，因此无法随意共享。
2. 即使开发者能够访问生产环境进行测试，也会遇到两个问题。
第一，**成本高、难以扩展**。开发者需要把应用部署到接近生产环境的基础设施中，才能观察真实资源竞争带来的影响。
第二，**生产环境本身是不断变化的**。随着 workload 的变化，资源竞争模式也会变化，测试条件难以复现。



现有合成方法及局限

1. Resource stressors

用 stress-ng、Intel MLC 等工具制造 CPU、内存、I/O 压力。

局限：过于粗粒度，通常只能拉满某类资源，难以模拟真实 workload 的动态变化。

2. Application benchmarks

用 SPEC、DCPerf、Fleetbench 等标准程序代表真实应用。

局限：覆盖场景有限，新应用类型需要重新设计 benchmark，扩展成本高。

3. Application cloning

先分析真实应用，再生成资源行为相似的合成程序。

局限：每个应用都要单独分析，成本高，且通常只能复现粗粒度统计特征

现有方法要么**太粗**，比如 stressor；要么**覆盖有限**，比如 benchmark；要么**成本太高**，比如 application cloning。它们都难以同时满足真实、可执行、可复现和低成本这几个要求。

研究背景与动机

现有合成方法及局限

1. Resource stressors

用 stress-ng、Intel MLC 等工具制造 CPU、内存、I/O 压力。

局限：过于粗粒度，通常只能拉满某类资源，难以模拟真实 workload 的动态变化。

2. Application benchmarks

用 SPEC、DCPerf、Fleetbench 等标准程序代表真实应用。

局限：覆盖场景有限，新应用类型需要重新设计 benchmark，扩展成本高。

3. Application cloning

先 profiling 真实应用，再生成资源行为相似的合成程序

局限

为了解决现有合成方法的局限，同时避免真实生产环境部署带来的高成本，提出了MIMESYS

现有

cloning。它们都难以同时满足真实、可执行、可复现和低成本这几个要求。

ation

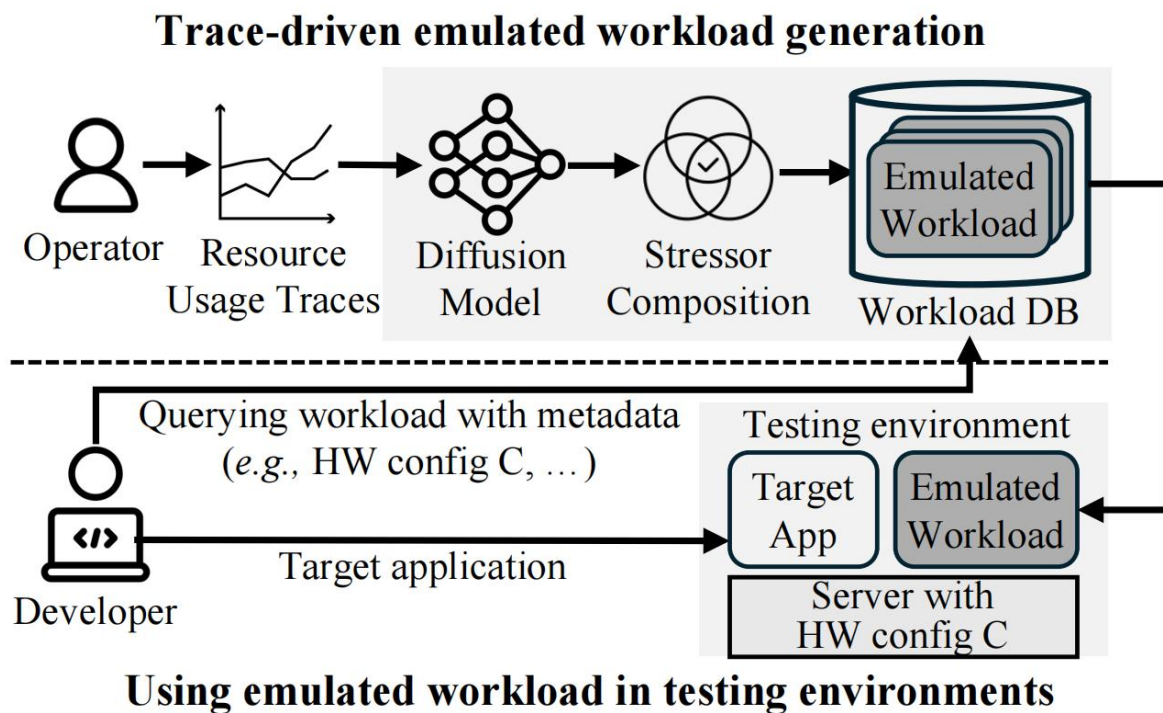
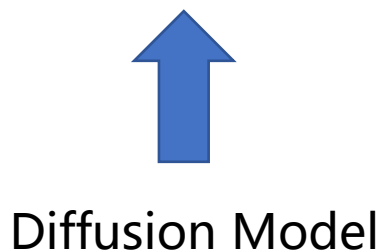


- 作者介绍
- 研究背景与动机
- **系统设计**
- 实验评估
- 总结与讨论

MIMESYS

Key Insight: 用合成的方式组成环境级 workload 来模拟真实的资源使用 trace，使它们在相同或相似硬件上运行时，产生类似的时间序列资源使用模式。

1. **组合Stressor**: 将模拟工作负载表示为参数化Stressor的组合。
2. **让模型学习资源使用 trace 到 Stressor 组合的映射**。通过模型来生成具体的组合Stressor。



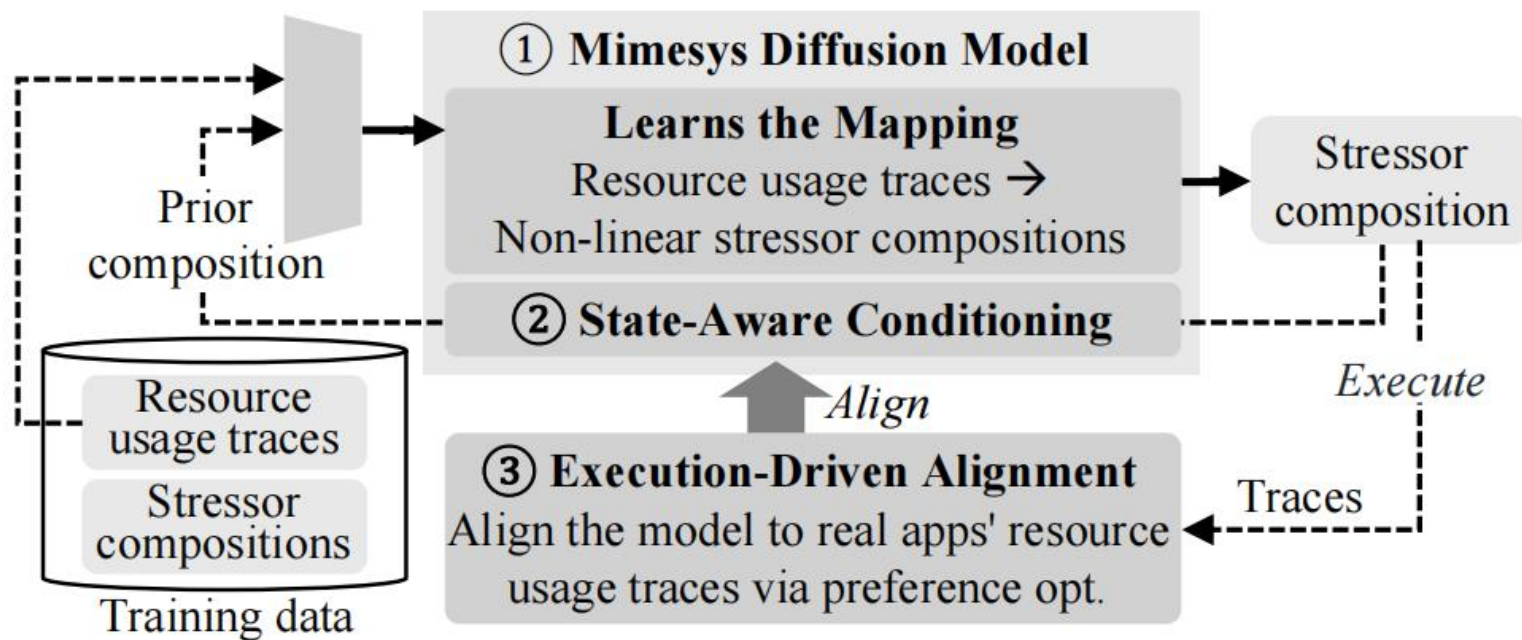


Challenge1: 时间和跨资源依赖难以建模。

同一个 **stressor 组合** 的效果会受**历史状态和并发负载影响**。例如缓存预热会让后续内存访问变快，CPU 竞争会改变线程调度和 cache miss。因此模型必须同时考虑时间上下文和多资源交互。

Challenge2: 合成数据与真实应用之间存在分布差距。

合成 stressor 数据能提供训练样本，**但真实应用有自己的执行节奏**。例如数据库会周期性查询、访存和刷盘，而随机 stressor 组合未必覆盖这种模式。因此模型需要用真实应用 trace 进行对齐，避免只学到“压力源分布”，而学不到“真实应用资源行为”。



C1: 时间和跨资源依赖难以建模。

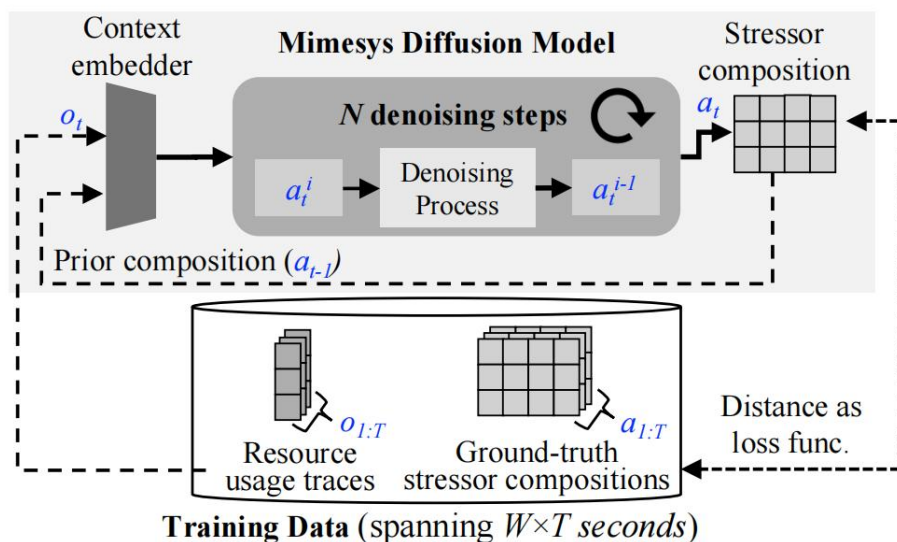
M1->C1: State-Aware Conditioning.

引入上一窗口 composition, 建模历史状态。传统方法只看当前窗口的 trace 进行预测。但 MIMESYS 还会看上一窗口的 composition。这样做是为了把历史状态带进来, 比如缓存预热、资源争用这些影响会跨时间窗口延续。

C2: 合成数据与真实应用之间存在分布差距。

M2->C2: Execution-Driven Alignment.

模型先根据真实应用 trace 生成 composition, 然后真的在硬件上执行, 得到新的 trace。系统比较生成 trace 和目标 trace 的差别, 越接近奖励越高, 再用强化学习更新模型。



1. 扩散模型适合处理这种多结果问题:

扩散模型从噪声开始，一步步去噪，逐渐生成结果。在这个过程中，它不是直接一次性预测最终 stressor 组合，而是逐步修正。

为什么选择diffusion model

trace 到 stressor composition 是**一对多的**。同一条资源使用轨迹，不一定只有一种压力源组合能产生。

比如想得到类似的内存带宽曲线，可能有几种办法：

- 4 个线程跑 memory stressor
- 8 个线程低强度跑 memory stressor
- memory stressor + cache stressor 混合跑

所以模型不能只输出一个“平均答案”，而要能从多个可能解中生成一个合理解。扩散模型适合这种多解生成问题。

2. 扩散模型能够从数据中学习非线性映射

压力源组合与资源使用之间不是简单线性关系。

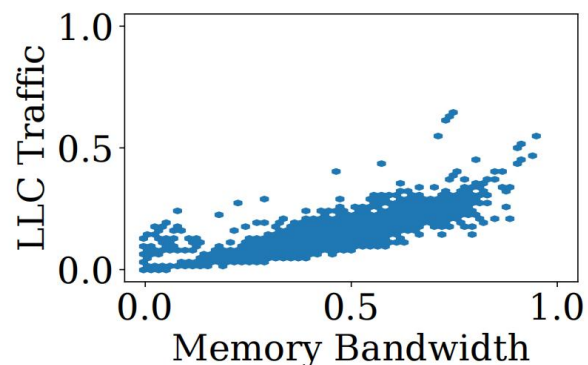
3. 扩散模型方便加入条件信息

它每一步都在做类似这样的事情：

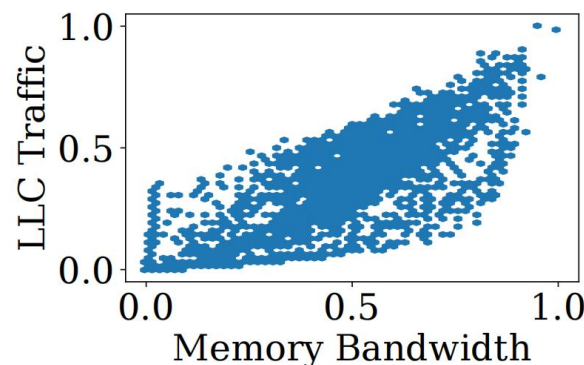
当前带噪结果 + 条件信息 → 更干净一点的结果

所以条件信息可以很自然地加进去，作为每一步去噪的指导。

训练数据的收集: Novelty-Guided Training Data Collection



(a) *Random sampling*



(b) *Novelty-guided collection*

如果只是随机生成很多 stressor 组合, 得到的数据会很偏。很多组合都会产生中等、普通、比较均衡的资源行为, 而稀有或极端模式很少出现。

已有数据

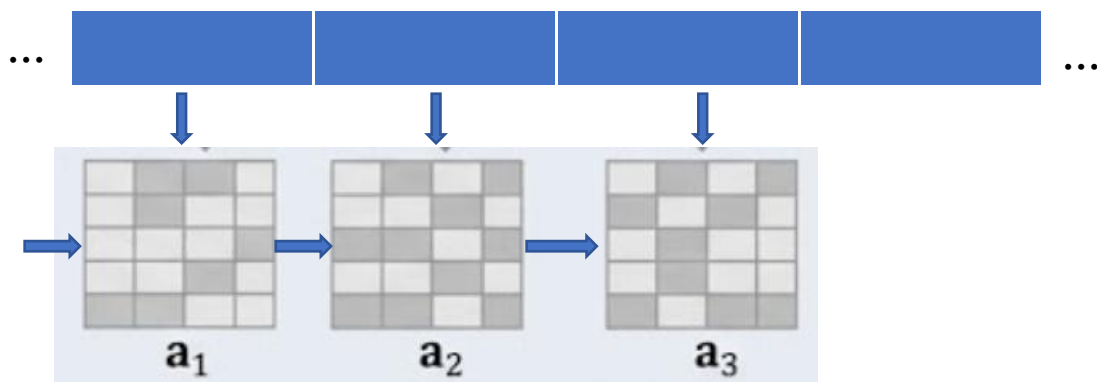
- > 训练一个轻量预测器
- > 预测候选 stressor组合 会产生什么资源 trace
- > 给候选打 novelty score
- > 优先执行 novelty 高的stressor组合
- > 采集真实 trace, 加入训练集
- > 继续迭代

novelty score:

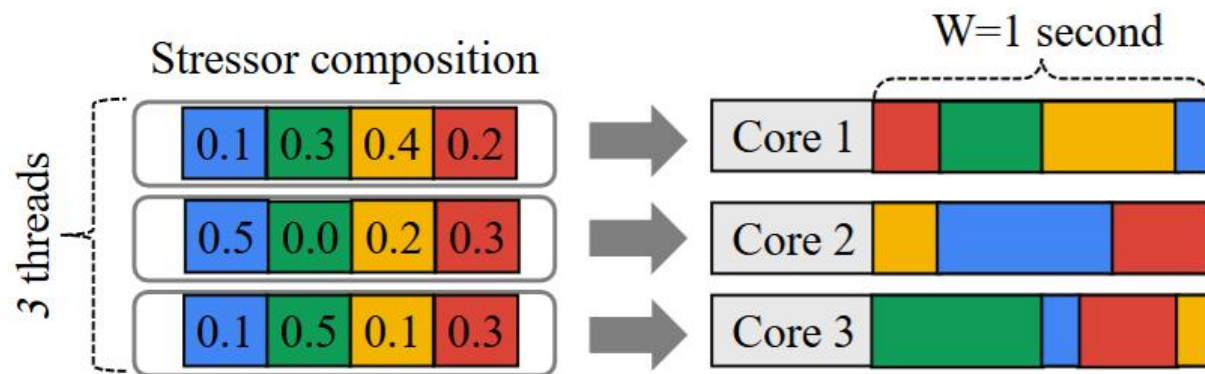
稀有性: 预测出的资源行为在当前数据集中是否少见;
不确定性: 预测器对这个组合的结果是否缺乏把握。

如何把目标 trace 转成一个真正能运行的仿真 workload

输入：一段目标应用的资源使用 trace



输出：各个时间窗口的composition



固定长度的时间窗口: MIMESYS 不是根据trace 一次性生成整个workload, 而是每 1 秒生成一个对应的 stressor 配方。

避免固定顺序带来的周期性时间偏置: 随机打乱每个时间窗口内 stressor 的执行顺序, 使生成的 workload 更接近真实应用的自然资源使用模式。



- 作者介绍
- 研究背景与动机
- GearDVFS系统设计
- **实验评估**
- 总结与讨论

实验设置

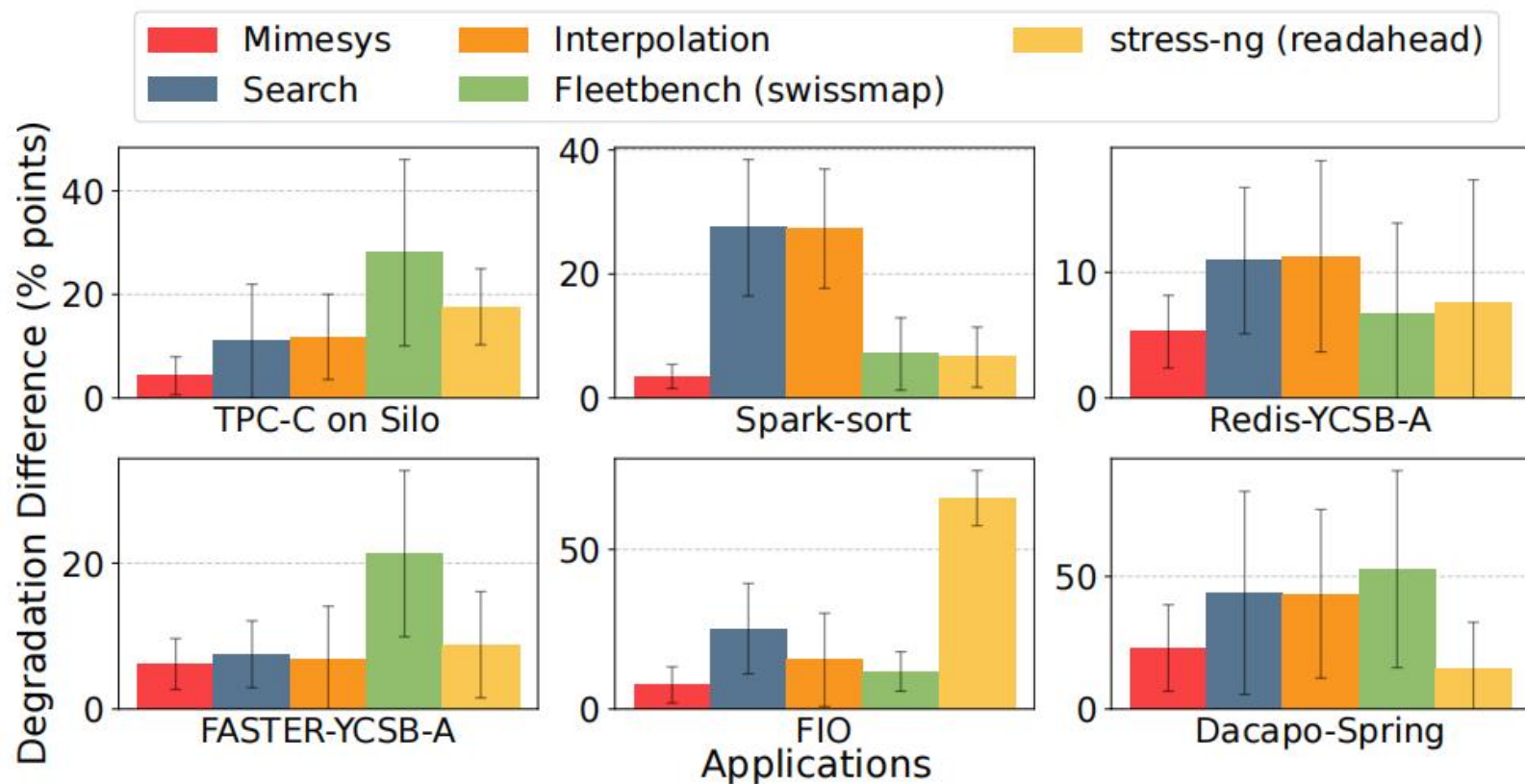
- **硬件平台:** CloudLab c220g2 机器, 使用 Intel Haswell 处理器, 20 个核心, 运行 Ubuntu 22.04 和 Linux kernel 5.15。
- **真实workload来源:**
 1. Web serving: DaCapo、Renaissance
 2. Big Data: HiBench on Spark
 3. KV Store: Redis、memcached、FASTER + YCSB
 4. ML Inference: MLPerf Inference
 5. Database: TPC-C、TPC-H
 6. Graph: GAP Benchmark。
- **输入 Trace: 23 维资源使用向量**
 1. 20 个 CPU core 的利用率
 2. 内存带宽使用量
 3. LLC 最后一级缓存流量
 4. 磁盘 I/O 吞吐量
- **Baseline:**
 1. Search-based: 直接从训练集中找最接近目标 trace 的 stressor composition
 2. Linear interpolation: 找最接近的若干训练样本, 然后插值得到 composition
 3. Single stressor: 单一资源压力源, 缓存、内存带宽使用 Fleetbench 的 swissmap, I/O 使用 stress-ng 的 readahead
- **评估指标:**
 1. Trace similarity: 用 DTW 衡量生成 trace 和真实 trace 的相似度。DTW 越低, 说明时间序列形状越接近。
 2. Application performance degradation: 比较目标应用和真实 workload 共部署时的性能下降, 和与 MIMESYS workload 部署时的性能下降是否一致 (throughput drop 或 latency increase)



实验评估

1. MIMESYS 生成的模拟工作负载，能不能像真实 workload 一样，对一起运行的应用造成相似的性能干扰？

6 个目标应用：数据库（Silo 上的 TPC-C）、键值存储（FASTER 和 Redis 上的 YCSB）、文件 I/O（FIO）、大数据（Spark Sort）和 Web 服务（DaCapo）

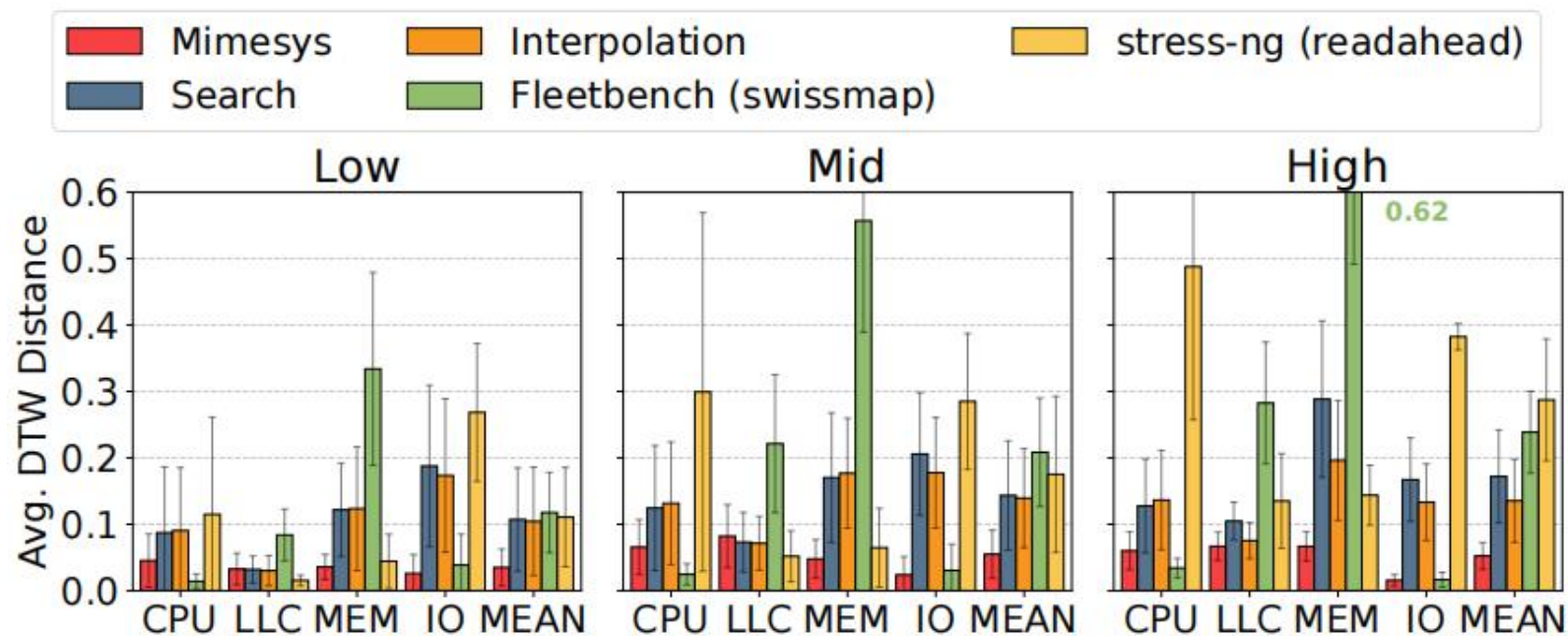


MIMESYS 在大多数应用上的差距都更小，平均只有 8.3%，而最好的 baseline 是 19.4 %。



产生的资源竞争效果更像真实 workload。

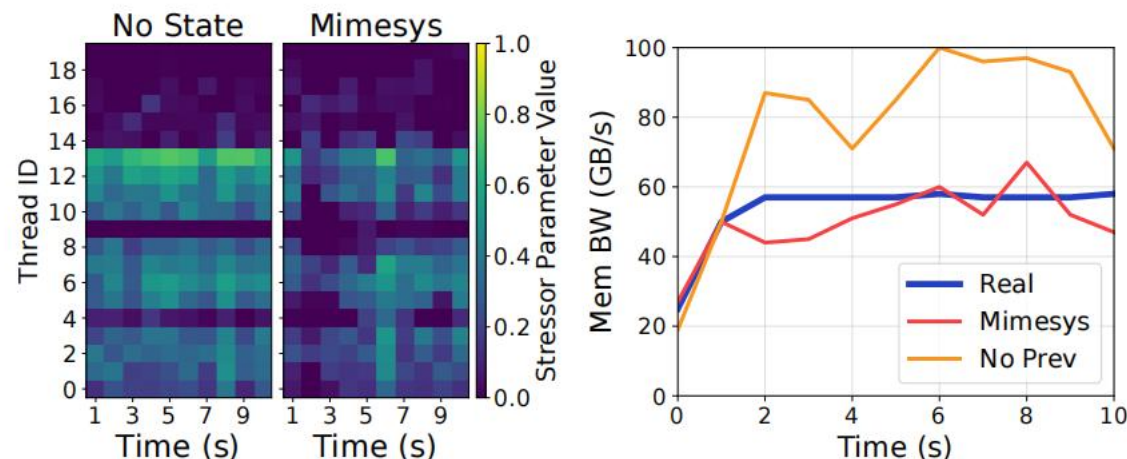
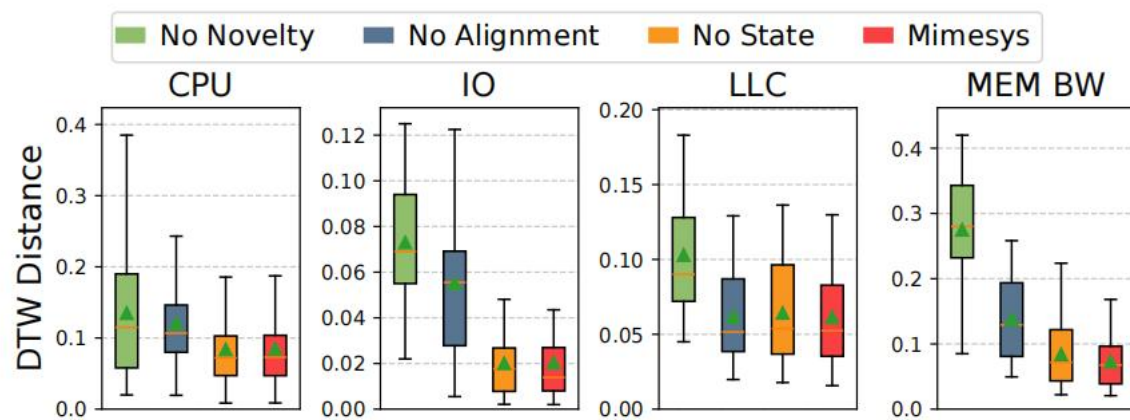
2. MIMESYS 生成的工作负载，其资源使用模式与真实应用有多接近？



DTW 是 Dynamic Time Warping。它是用来比较两条资源使用时间序列“形状是否相似”的方法。

MIMESYS 在大多数指标和负载等级下 DTW 都最低，说明生成出来的资源曲线最接近真实 workload。

3. 消融实验：MIMESYS 里面几个关键设计到底有没有用？



- 完整版 MIMESYS
- 去掉 novelty-guided data collection
- 去掉 execution-driven alignment
- 去掉 state-aware conditioning

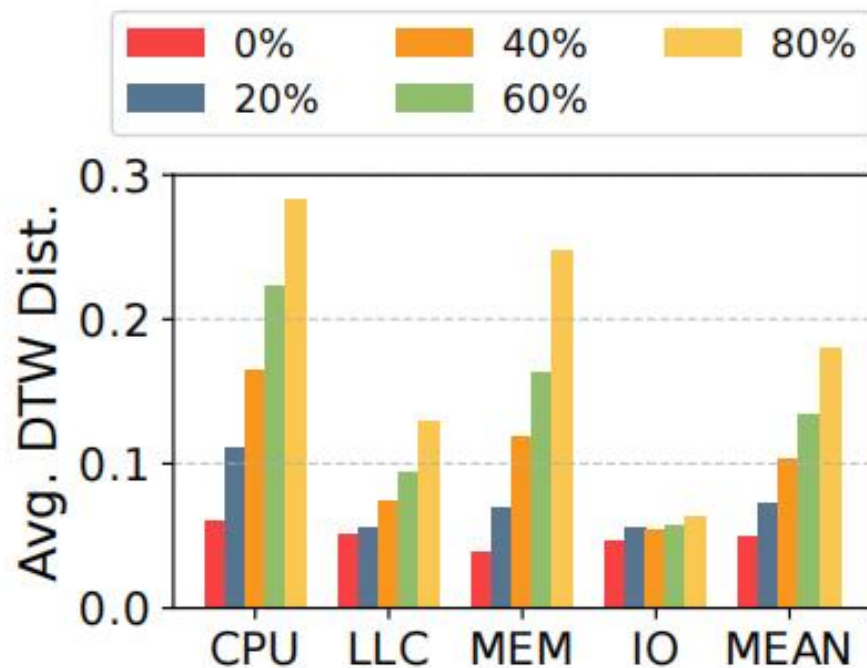
考虑历史状态后：

- MIMESYS 会根据上一时间窗口的执行状态调整当前压力。
- 生成的 memory bandwidth 曲线更接近真实 workload。

MIMESYS 的三个关键设计都有效，去掉任何一个都会让生成 trace 和真实 trace 的差距变大。

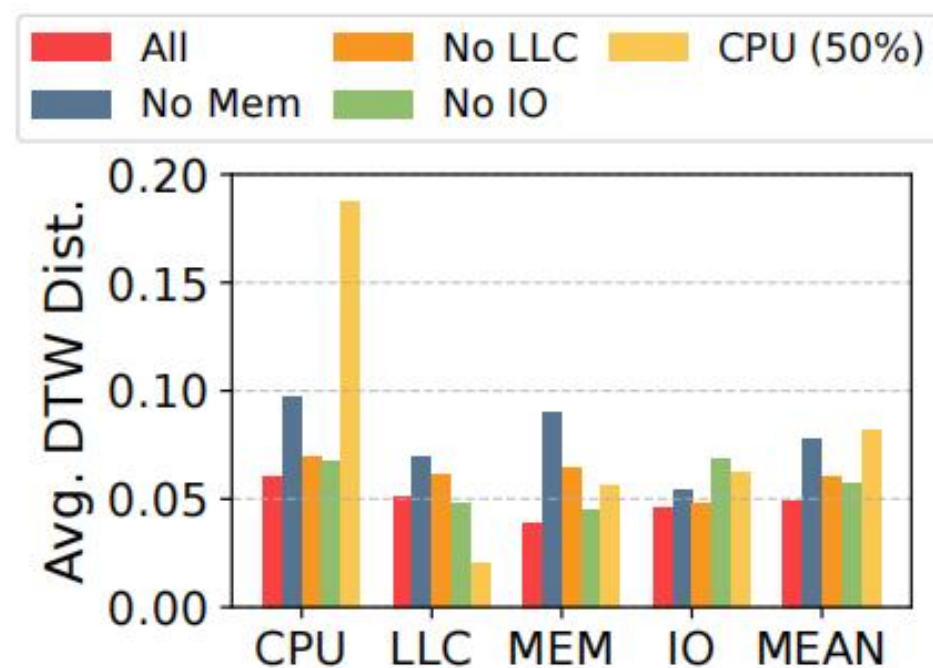
实验评估

4. 鲁棒性实验：如果输入 trace 缺失一部分，或者缺少某类资源指标，MIMESYS 是否仍然能生成合理 workload?



(a) *Varying trace dropping rate*

随机丢掉一部分 trace，模型仍然是逐渐退化



(b) *Varying dropped metric*

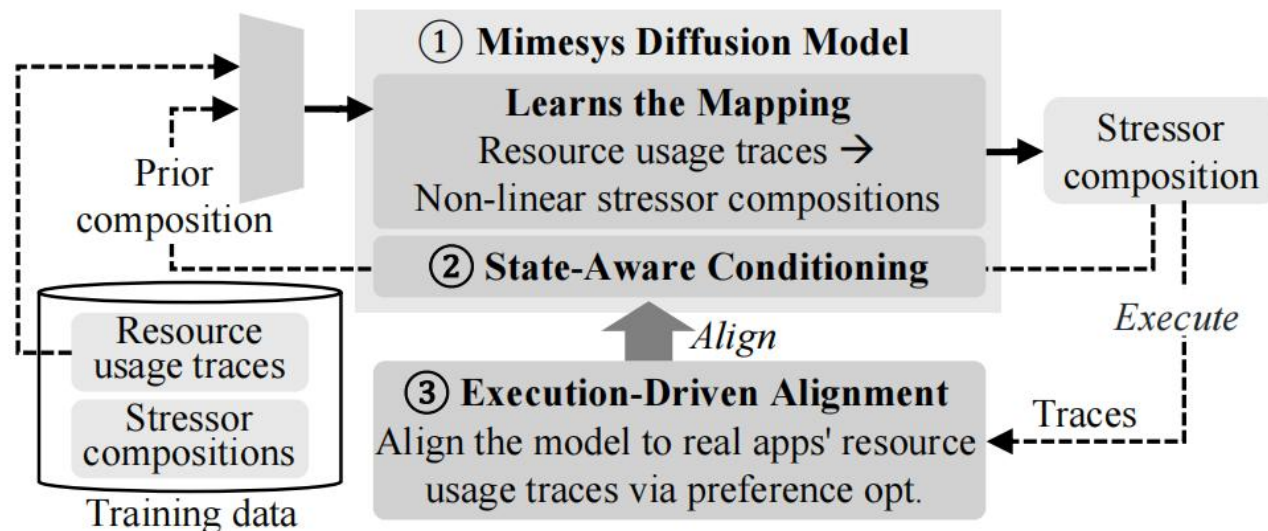
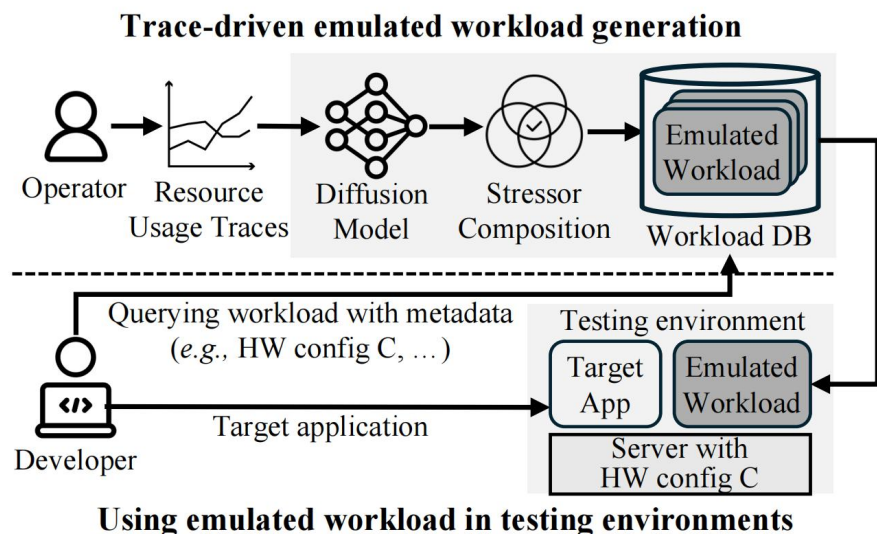
虽然去掉某一种资源指标，模型仍能利用其他资源指标生成比较合理的压力源组合



- 作者介绍
- 研究背景与动机
- GearDVFS系统设计
- 实验评估
- **总结与讨论**

总结与讨论

总结:



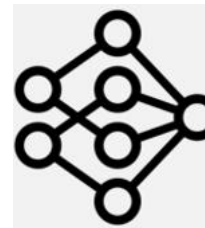
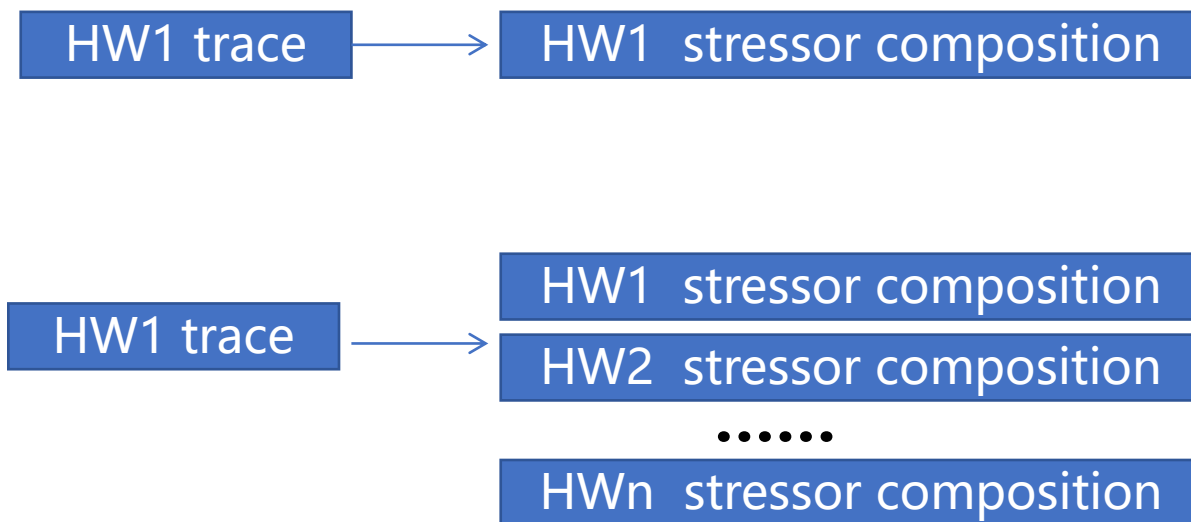
- 本文提出了 MIMESYS，一个从真实应用的 resource usage trace 生成可执行模拟 workload 的系统。
- 解决的问题是：真实生产环境很难直接拿来测试，因为涉及隐私、专有代码和复杂依赖；而目前的 baseline 又不够真实，难以复现真实共置环境下的资源竞争。

核心设计:

1. State-aware conditioning: 引入上一窗口 composition，建模历史状态
2. Execution-driven alignment: 根据生成轨迹和目标轨迹的距离给 reward，用强化学习方式微调模型
3. Novelty-guided data collection: 用一个轻量预测器挑选“可能覆盖新资源行为”的组合去执行和采样

1. 这篇文章能不能进一步提高:

MIMESYS 已经能从 trace 生成可执行 workload, 那能不能进一步支持跨硬件平台? 能不能减少训练数据采集成本?



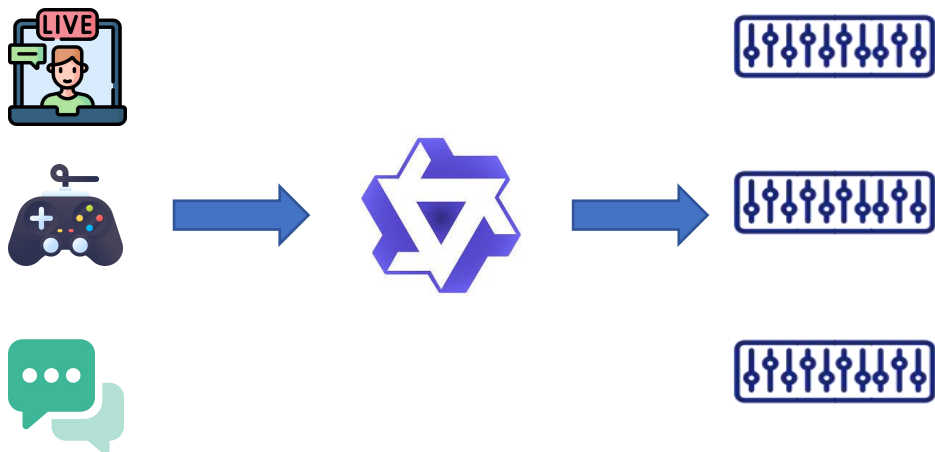
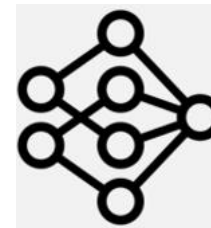
- 在输入中加入 hardware profile, 例如 core 数、I/O、memory bandwidth;
- 训练 hardware-aware diffusion model;

总结与讨论

2. 别人的好方法能不能用到我们的场景：

MIMESYS采用了扩散模型针对 trace 到 stressor composition 这种一对多问题。

我的场景： 利用自回归模型为手机上不同app场景生成配置

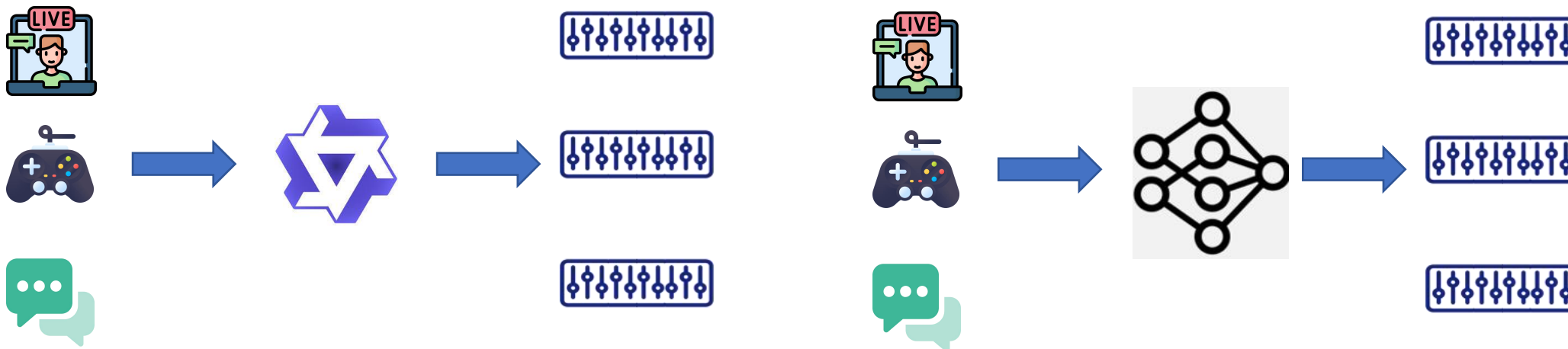


总结与讨论

2. 别人的好方法能不能用到我们的场景：

MIMESYS采用了扩散模型针对 trace 到 stressor composition 这种一对多问题。

我的场景： 利用自回归模型为手机上不同app场景生成配置

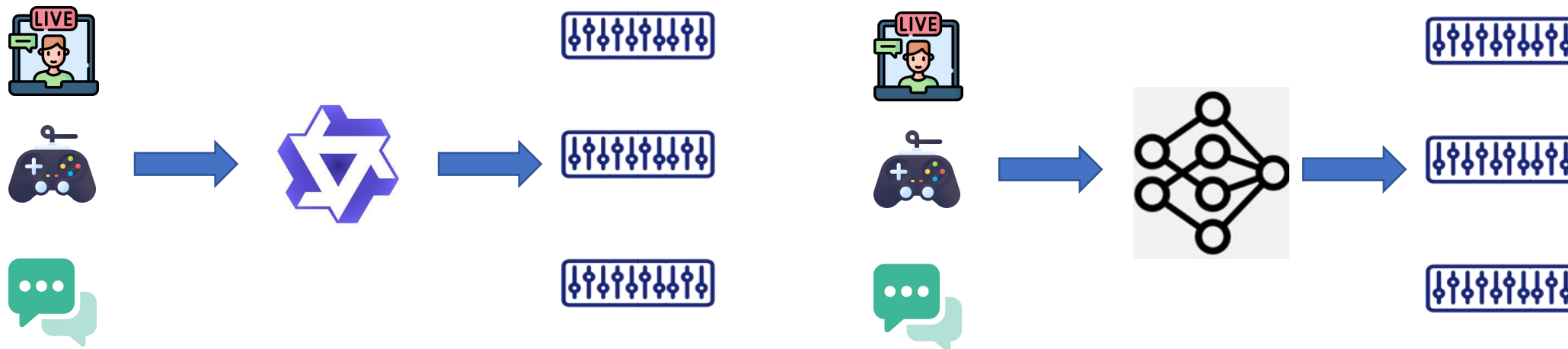


总结与讨论

2. 别人的好方法能不能用到我们的场景：

MIMESYS采用了扩散模型针对 trace 到 stressor composition 这种一对多问题。

我的场景： 利用自回归模型为手机上不同app场景生成配置



这篇文章选择扩散模型的原因：

1. 扩散模型适合处理这种多结果问题
2. 扩散模型可以从大量样本中学习复杂的非线性映射
3. 扩散模型方便加入条件信息

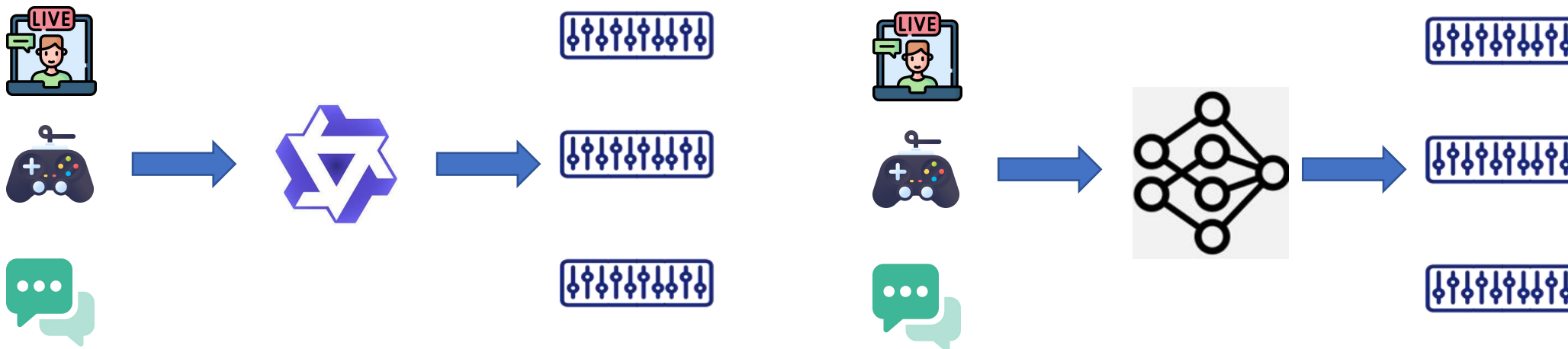
1. 加入上一个场景的信息
2. 当前的负载状态信息

总结与讨论

2. 别人的好方法能不能用到我们的场景：

MIMESYS采用了扩散模型针对 trace 到 stressor composition 这种一对多问题。

我的场景： 利用自回归模型为手机上不同app场景生成配置



这篇文章选择扩散模型的原因：

1. 扩散模型适合处理这种多结果问题
2. 扩散模型可以从大量样本中学习复杂的非线性映射
3. 扩散模型方便加入条件信息

1. 加入上一个场景的信息
2. 当前的负载状态信息



Q&A

2026年7月6日

冯敏远